ROTEIRO PRÁTICO 04 • PROGRAMAÇÃO PARA A INTERNET 2

Arquitetura Modular com ES6 Modules

Vitrine Comunitária em camadas: services, views e utils, com modais, toasts e validação

Módulo 2 | ES6 Modules · Bootstrap 5 · Live Server | Pré-requisito: Roteiro 03

Unidade Curricular
Programação para a Internet 2
CST em Sistemas para a Internet

Docente
Prof. André Moraes
IFSC Câmpus Garopaba

Produto deste roteiro
Vitrine Comunitária de Serviços modular, com cadastro em modal, edição, exclusão e filtros

Ao final desta prática você será capaz de...

- Superar a arquitetura monolítica de arquivo único (`app.js`), organizando o frontend em múltiplos módulos especializados utilizando **ES6 Modules** (`import` e `export`);
- Declarar o ponto de entrada modular na página web através da diretiva: `<script type="module" src="js/main.js"></script>`;
- Compreender e aplicar a separação estrita de responsabilidades no cliente em três camadas:
  - **Camada de Serviços (`services/`):** Regras de negócio, cálculos e persistência pura no `localStorage` sem acoplamento ao DOM;
  - **Camada de Visão (`views/`):** Renderização de cards, tabelas, modais e Toasts via *template literals* do Bootstrap 5;
  - **Camada Utilitária (`utils/`):** Funções puras reutilizáveis de formatação (moeda brasileira, telefone, máscaras);
  - **Orquestrador Central (`main.js`):** Captura de eventos do usuário e coordenação entre camadas.
- Implementar o **CRUD Completo** (*Create, Read, Update, Delete*) com persistência em armazenamento local;
- Implementar **validação visual no cliente** com feedback imediato ao usuário utilizando as classes nativas do Bootstrap 5 (`.was-validated` e `.invalid-feedback`);
- Disparar notificações flutuantes dinâmicas (**Bootstrap Toasts**) para confirmar operações de cadastro, alteração e exclusão;
- Implementar filtragem reativa em tempo real por categoria de serviço na vitrine comunitária.

Mapa da aula

1 Estrutura e componentes
Pastas do projeto, modais, toasts e cards (Seções 3 a 4)

2 ES6 Modules
`type="module"`, `export` e `import` (Seção 5)

3 Construir em 5 fases
HTML, utils, services, views e main.js (Seção 6)

4 Conferir e publicar
Checklist e critérios avaliativos (Seção 7)

1 Onde este roteiro se encaixa



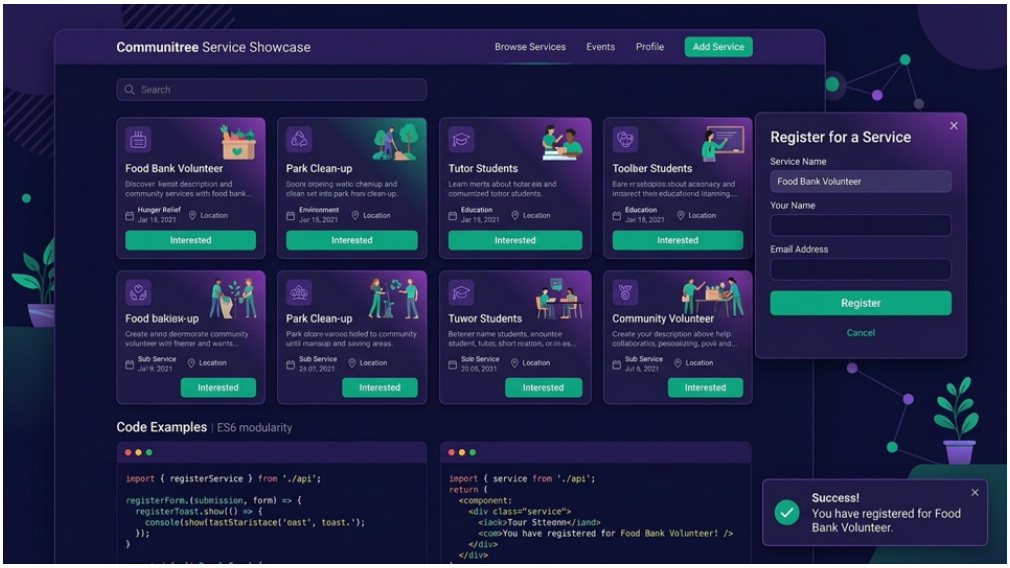


Figura 1: Visão Geral da Aplicação: Vitrine Comunitária de Garopaba com arquitetura modular em camadas ES6, modais e Toasts.

2 Introdução e Contextualização do Projeto

Nos Roteiros 1, 2 e 3, trabalhamos com scripts JavaScript concentrados em um único arquivo (`app.js`). Embora essa abordagem seja excelente para aprender os fundamentos, ela se torna inviável em aplicações reais de médio e grande porte, onde a mistura de seletores do DOM, regras de validação e persistência gera o chamado *"código espaguete"*.

No **Roteiro Prático 04**, damos o passo definitivo rumo à arquitetura profissional de software no frontend. Desenvolveremos a **Vitrine Comunitária de Empreendedores Locais de Garopaba**, uma aplicação que permite cadastrar, listar, filtrar, editar e gerenciar serviços da comunidade local.

Este projeto serve também como a **base tecnológica inicial para a Ação Curricularizada de Extensão na SNCT (Semana Nacional de Ciência e Tecnologia)**.

3 Estrutura Arquitetural de Pastas

O projeto deve ser organizado na pasta `roteiro-4/exercicio-roteiro4/` seguindo rigorosamente a estrutura modular:

Árvore de Diretórios Modular do Projeto

```
roteiro-4/exercicio-roteiro4/
|-- index.html     <-- Vitrine e forms (BS5)
|-- styles.css     <-- Estilos e transições
`-- js/
    |-- main.js    <-- Orquestrador central
    |-- services/
    |   `-- vitrineService.js <-- CRUD Storage
    |-- views/
    |   `-- vitrineView.js    <-- HTML dinâmico
    `-- utils/
        `-- formatters.js     <-- Formatações
```

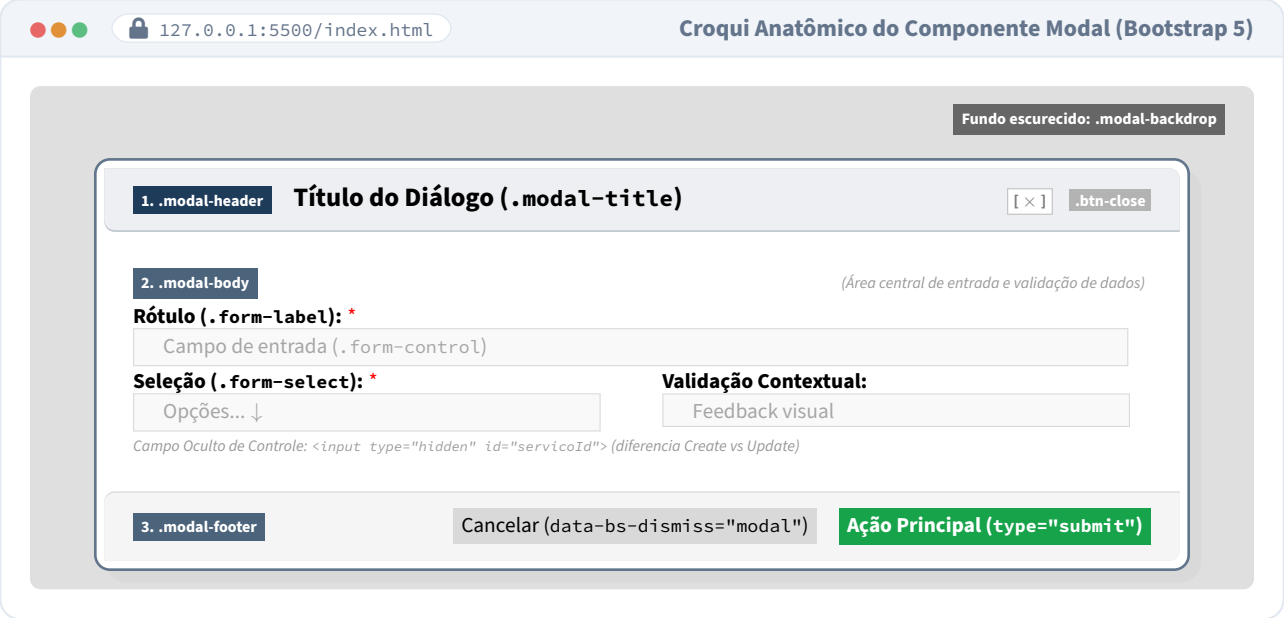
4 Fundamentação Teórica: Componentes Modernos de Interface (Bootstrap 5)

Antes de iniciarmos a implementação prática guiada, é fundamental compreender a finalidade, o comportamento e a anatomia dos principais componentes de interface gráfica utilizados na aplicação:

4.1 Janelas Modais (*Modals*): Diálogos Sobrepostos de Alta Atenção

Uma **janela modal** (ou diálogo modal) é um elemento flutuante que surge acima de todo o conteúdo da página, bloqueando temporariamente a interação com a tela de fundo por meio de uma camada escurecida semi-transparente chamada ***backdrop***.

- **Finalidade de Uso:** Concentrar a atenção exclusiva do usuário em uma tarefa crítica (como preencher o formulário de cadastro ou atualizar dados de um serviço) sem sair da tela atual nem recarregar o navegador (*Single Page Application*).



Anatomia no Bootstrap 5 (conforme ilustrado acima):

Elemento / Classe	Camada Anatômica	Papel Arquitetural e Comportamento
`.modal`	Contêiner Base	Invisível por padrão (`display: none`), ativa a transição animada suave (`.fade`) e o bloqueio de tela (*backdrop*).
`.modal-dialog`	Geometria & Posição	Controla largura responsiva e o alinhamento vertical no viewport via `.modal-dialog-centered`.
`.modal-content`	Invólucro do Diálogo	Superfície branca com cantos arredondados e elevação (`.shadow-lg`), estruturada em 3 seções:
`.modal-header`	1. Cabeçalho	Título da ação (`.modal-title`) e botão de fechar nativo (`.btn-close` com `data-bs-dismiss="modal"`).
`.modal-body`	2. Corpo Central	Área nobre do formulário com rótulos, campos de entrada, validações e campo oculto de controle (`#servicoId`).
`.modal-footer`	3. Rodapé de Ações	Área de decisão e submissão com botões de controle (*Cancelar* e *Salvar*).
`bootstrap.Modal`	Controle JavaScript	Instanciação programática (`getOrCreateInstance`) para abertura (`.show()`) e fechamento (`.hide()`).

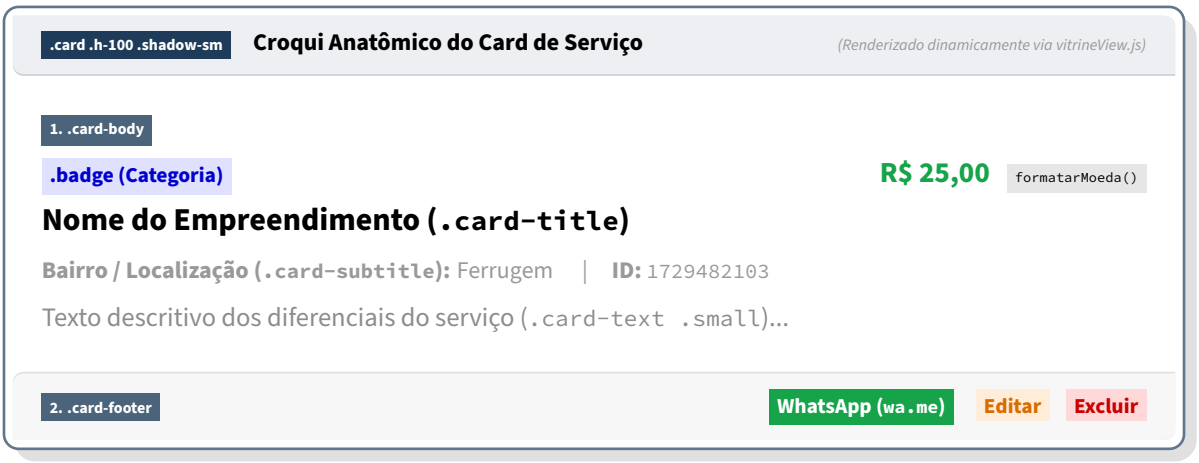
4.2 Notificações Flutuantes (*Toasts*): Feedback Assíncrono Não-Intrusivo

O componente **Toast** é uma notificação compacta e temporária que surge sobreposta em uma posição fixa da tela (canto superior direito) para confirmar o sucesso de operações (ex: *"Serviço cadastrado com sucesso!"*).

- **Vantagem sobre o `alert()`:** O diálogo nativo `window.alert()` é síncrono e bloqueante: ele congela a execução de qualquer outro script JavaScript no navegador e força o usuário a clicar em "OK" para continuar. O **Toast**, por outro lado, é assíncrono, suave e desaparece automaticamente após alguns segundos (*autohide*), proporcionando uma experiência de usuário profissional;
- **Posicionamento com `.toast-container`:** O elemento contêiner utiliza classes utilitárias de posicionamento fixo (`position-fixed top-0 end-0 p-3`) com índice z prioritário (`z-index: 1090`), garantindo visibilidade imediata sem deslocar os demais elementos do layout.

4.3 Cartões de Conteúdo (*Cards*): Unidades Modulares e Grid Uniforme

Um **Card** é um contêiner retangular flexível (`.card .shadow-sm`) que agrupa de forma coesa todas as informações e ações pertinentes a um único registro:



Anatomia no Bootstrap 5 (conforme ilustrado acima):

Elemento / Classe	Camada Anatômica	Papel Arquitetural e Comportamento
`.card`	Contêiner Modular	Unidade visual básica com borda suave e elevação sutil (`.shadow-sm`) que agrupa os dados de um serviço.
`.h-100`	Equalização Vertical	Força 100% da altura da coluna no grid Flexbox, garantindo nivelamento e alinhamento visual uniforme na vitrine.
`.card-body`	1. Corpo Principal	Abriga a badge de categoria, o título comercial (`.card-title`), o preço formatado, a localização e a descrição.
`.card-footer`	2. Rodapé de Ações	Fundo neutro transparente (`.bg-transparent`) agrupando os botões de ação: WhatsApp, Editar e Excluir.

4.4 Validação Visual e o Campo Oculto (`type="hidden"`)

- **Atributo `novalidate`:** Informa ao navegador para suprimir os balões de validação nativos, permitindo que a aplicação assuma o controle visual das mensagens de erro com o Bootstrap: ao adicionar a classe `.was-validated` ao formulário, cada campo é colorido conforme o resultado da validação nativa do HTML5 (verde para válido, vermelho para inválido) e os contêineres contextuais `.invalid-feedback` dos campos inválidos são exibidos;
- **Papel Arquitetural do Campo Oculto (`#servicoId`):** O campo com `type="hidden"` não é renderizado visualmente para o usuário, mas é o elemento-chave que diferencia as operações de **Criação** (*Create*) e **Atualização** (*Update*) no padrão CRUD: se o campo estiver vazio, o clique em "Salvar" cria um novo serviço; se contiver um ID, atualiza o serviço correspondente mantendo o identificador original.

5 Fundamentação Teórica: Arquitetura Modular com ES6 Modules

5.1 Declaração no HTML (`type="module"`)

Ao declarar `<script type="module" src="js/main.js"></script>`, o navegador aplica comportamentos modernos essenciais:

- **Modo Estrito Automático (`"use strict"`):** Impede o uso de variáveis não declaradas e falhas silenciosas;
- **Escopo Fechado de Módulo:** Variáveis e funções criadas dentro de um arquivo não poluem o escopo global (`window`);
- **Carregamento Assíncrono com Adiamento (`defer`):** O script aguarda o parsing completo do DOM antes de executar.

5.2 Exportação e Importação (`export` e `import`)

Para compartilhar recursos entre arquivos, usamos a sintaxe explícita de módulos:

Exemplo de Exportação Nomeada e Importação:

JavaScript

```
// No arquivo js/utils/formatters.js
export function formatarTelefone(telefone) {
  return telefone.replace(
    /^(\d{2})(\d{5})(\d{4})/,
    '($1) $2-$3'
  );
}

// No arquivo js/main.js
import { formatarTelefone }
  from './utils/formatters.js';

// Resultado: (48) 99999-8888
console.log(formatarTelefone('48999998888'));
```

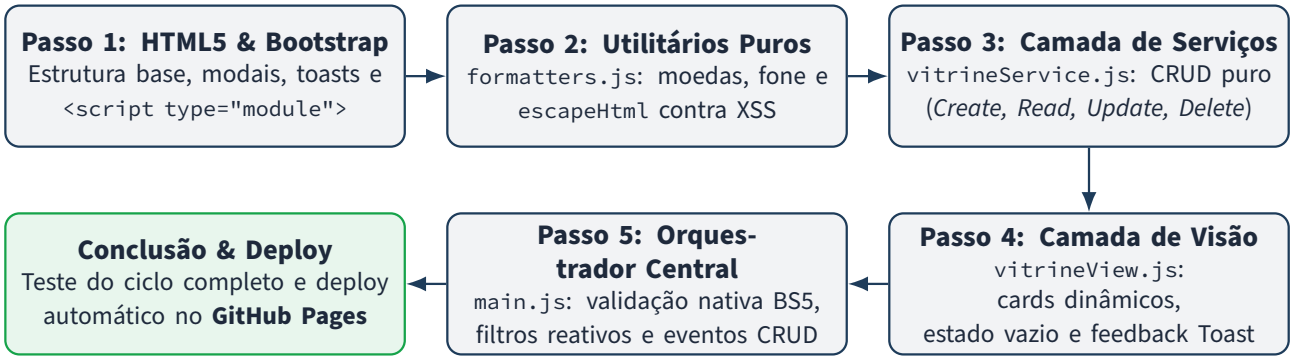


Figura 2: Pipeline arquitetural e sequência cronológica de implementação do Roteiro 04.

Links diretos para o GitHub (Código-Fonte de Referência):

Caso prefira consultar, clonar ou comparar os arquivos diretamente no repositório:

- **index.html** → https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-internet-2/blob/main/roteiro4-novamente/index.html
- **main.js (orquestrador)** → https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-internet-2/blob/main/roteiro4-novamente/js/main.js
- **styles.css** → https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-internet-2/blob/main/roteiro4-novamente/styles.css
- **formatters.js** → https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-internet-2/blob/main/roteiro4-novamente/js/utils/formatters.js
- **vitrineService.js** → https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-internet-2/blob/main/roteiro4-novamente/js/services/vitrineService.js
- **vitrineView.js** → https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-internet-2/blob/main/roteiro4-novamente/js/views/vitrineView.js

6 Passo a Passo Guiado do Roteiro

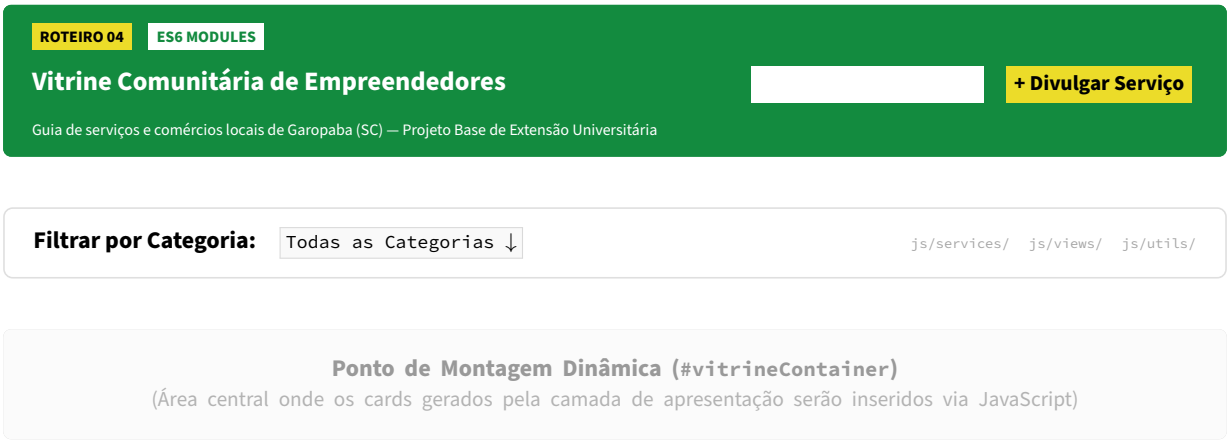
Antes de começar:

- **Abra sempre pelo Live Server.** Módulos ES6 não carregam quando o `index.html` é aberto com duplo clique (endereço `file://`).
- **Ao copiar código deste PDF**, copie uma página por vez. Quando um bloco continua na página seguinte, o cabeçalho e o rodapé do documento vêm junto na seleção e precisam ser apagados.
- **Código no GitHub:** Os links diretos para cada arquivo no repositório estão disponíveis no quadro da página anterior.

Fase 1.1/5: Estrutura HTML5 Base e Ponto de Entrada (index.html)

Responsabilidade Arquitetural: Estabelecer o esqueleto visual da página, carregar as dependências de estilo (Bootstrap 5 e Icons), fornecer os pontos de ancoragem da interface (`#vitrineContainer`, `#toastNotificacao`, `#totalServicosBadge` e `#filtroCategoria`) e conectar o módulo raiz `js/main.js`.

127.0.0.1:5500/index.html — **Interface Base: aparência final (após os Passos 1.3 e 5)**



index.html — HTML

```html
<!-- index.html (Parte 1: Estrutura Base e Scripts) -->
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vitrine Comunitária de Garopaba — Roteiro 04 (ES6 Modules)</title>
  <!-- Bootstrap 5 CSS, Icons & Google Fonts -->
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
```

```
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link rel="stylesheet"
    href="https://fonts.googleapis.com/css2?family=Outfit:wght@500;700;800&display=swap">
  <link rel="stylesheet"
  href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700&display=swap">
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <!-- Container de Toasts com corpo #toastMensagem -->
  <div class="toast-container position-fixed top-0 end-0 p-3" style="z-index: 1090;">
    <div id="toastNotificacao"
         class="toast align-items-center text-bg-success border-0 shadow-lg"
         role="alert">
      <div class="d-flex">
        <div class="toast-body d-flex align-items-center gap-2">
          <i class="bi bi-check-circle-fill fs-5"></i>
          <div>
            <strong id="toastTitulo" class="d-block">Sucesso!</strong>
            <span id="toastMensagem">Operação realizada com êxito.</span>
          </div>
        </div>
        <button type="button" class="btn-close btn-close-white me-2 m-auto"
                data-bs-dismiss="toast"></button>
      </div>
    </div>
  </div>

  <!-- Hero Banner com Tipografia Outfit e Badges -->
  <header class="hero-banner shadow-sm">
    <div class="container">
      <div class="d-flex align-items-center justify-content-between flex-wrap gap-3">
        <div>
          <div class="d-flex align-items-center gap-2 mb-2">
            <span class="badge bg-warning text-dark font-outfit fw-bold px-3 py-2 rounded-pill">
              <i class="bi bi-code-slash me-1"></i> ROTEIRO PRÁTICO 04
            </span>
            <span class="badge bg-white text-success font-outfit fw-bold px-3 py-2 rounded-pill">
              ES6 MODULES &amp;amp; ARQUITETURA EM CAMADAS
            </span>
          </div>
          <h1 class="display-6 fw-bold font-outfit mb-1">Vitrine Comunitária de Empreendedores</h1>
          <p class="mb-0 opacity-75 small">
            Guia de serviços e comércios locais de Garopaba (SC) —
            Projeto Base de Extensão Universitária
          </p>
        </div>
        <div class="d-flex align-items-center gap-2">
          <span class="badge-counter" id="totalServicosBadge">Carregando...</span>
          <button data-bs-toggle="modal" data-bs-target="#modalCadastro"
```

```
            class="btn btn-warning text-dark font-outfit fw-bold px-3 py-2 rounded-pill shadow-sm">
            <i class="bi bi-plus-circle-fill me-1"></i> Divulgar Serviço
          </button>
        </div>
      </div>
    </div>
  </header>

  <main class="container mb-5">
    <!-- Barra de Controle e Filtros -->
    <div class="card border-0 shadow-sm rounded-4 mb-4 p-3 bg-white">
      <div class="row g-3 align-items-center justify-content-between">
        <div class="col-md-5">
          <div class="input-group">
            <span class="input-group-text bg-light border-end-0">
              <i class="bi bi-funnel-fill text-muted"></i>
            </span>
            <select class="form-select border-start-0 bg-light" id="filtroCategoria">
              <option value="todas">Todas as Categorias</option>
              <option value="Alimentação">Alimentação &amp;amp; Gastronomia</option>
              <option value="Tecnologia">Tecnologia &amp;amp; Internet</option>
              <option value="Artesanato">Artesanato &amp;amp; Arte Local</option>
              <option value="Serviços Gerais">Serviços Gerais &amp;amp; Reformas</option>
              <option value="Turismo">Turismo &amp;amp; Passeios</option>
            </select>
          </div>
        </div>

        <div class="col-md-7 text-md-end text-muted small">
          <span class="badge bg-light text-dark border me-1">
            <i class="bi bi-folder-fill text-warning me-1"></i> js/services/
          </span>
          <span class="badge bg-light text-dark border me-1">
            <i class="bi bi-layout-text-window-reverse text-info me-1"></i> js/views/
          </span>
          <span class="badge bg-light text-dark border">
            <i class="bi bi-gear-wide-connected text-success me-1"></i> js/utils/
          </span>
        </div>
      </div>
    </div>

    <!-- Ponto de Montagem dos Cards Dinâmicos -->
    <div class="row" id="vitrineContainer"></div>
  </main>

  <!-- Bootstrap Bundle JS + Entrada Modular ES6 -->
  <script
    src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
  <script type="module" src="js/main.js"></script>
</body>
```

```
</html>
```

Checkpoint de Validação 1.1: Como Testar no Navegador

Abra o `index.html` no navegador via **Live Server**:

1. Título, badges, contador (*"Carregando..."*) e botão **"Divulgar Serviço"** visíveis (o degradê verde do banner só surgirá com o `styles.css` no Passo 1.3);
2. Seletor de categorias locais e badges dos módulos visíveis e funcionais;
3. Área central vazia, pronta para receber os cards dinâmicos via JavaScript.

Nota: Erros 404 no Console para `styles.css` e `js/main.js` são esperados até os Passos 1.3 e 5. O modal abrirá após estruturado no Passo 1.2.

Fase 1.2/5: Formulário Modal de Cadastro e Edição (index.html)

Responsabilidade Arquitetural: Fornecer a interface de diálogo acessível para inserção e atualização de serviços, contendo validações visuais nativas do HTML5 (`required`), seletores padronizados e um campo oculto (`#servicoId`) que diferencia operações de *Create* (novo) e *Update* (edição).

Onde e como inserir este código no `index.html`:

- **Atenção: Não apague nem substitua** o código estruturado no Passo 1.1! Todo o documento construído anteriormente permanece intacto.
- **Local exato de inserção:** Insira o bloco completo do modal logo após a tag de fechamento `</main>` e imediatamente antes das tags de script (`<!-- Bootstrap Bundle JS... -->`).

127.0.0.1:5500/index.html — **Navegador Web: index.html (Diálogo Modal Ativo)**



index.html — HTML

```
<!-- index.html (Inserir logo após </main> e imediatamente antes de <script>) -->
<!-- ========================================================================== -->
<!-- PARTE 2: MODAL DE CADASTRO E EDIÇÃO (Não apaga a Parte 1!)                 -->
<!-- ========================================================================== -->
<div class="modal fade" id="modalCadastro" tabindex="-1" aria-hidden="true">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content rounded-4 border-0 shadow-lg">
      <div class="modal-header border-0 pb-0">
        <h5 class="modal-title font-outfit fw-bold text-dark" id="modalCadastroLabel">
          <i class="bi bi-shop text-success me-2"></i>Cadastrar Serviço na Vitrine
        </h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"
                aria-label="Fechar"></button>
      </div>
      <form id="formCadastro" novalidate>
```

```
        <input type="hidden" id="servicoId" value="">
        <div class="modal-body pt-3">
          <p class="text-muted small mb-3">Preencha as informações do empreendedor local:</p>

          <div class="mb-3">
            <label for="nome" class="form-label small fw-bold text-muted">
              Nome do Empreendimento *
            </label>
            <input type="text" id="nome" class="form-control rounded-3" required
                   placeholder="Ex: Maré Alta Artesanatos">
            <div class="invalid-feedback">Por favor, informe o nome do empreendimento.</div>
          </div>

          <div class="row g-2 mb-3">
            <div class="col-md-6">
              <label for="categoria" class="form-label small fw-bold text-muted">Categoria *</label>
              <select id="categoria" class="form-select rounded-3" required>
                <option value="" selected disabled>Selecione...</option>
                <option value="Alimentação">Alimentação &amp;amp; Gastronomia</option>
                <option value="Tecnologia">Tecnologia &amp;amp; Internet</option>
                <option value="Artesanato">Artesanato &amp;amp; Arte Local</option>
                <option value="Serviços Gerais">Serviços Gerais &amp;amp; Reformas</option>
                <option value="Turismo">Turismo &amp;amp; Passeios</option>
              </select>
              <div class="invalid-feedback">Selecione uma categoria.</div>
            </div>
            <div class="col-md-6">
              <label for="bairro" class="form-label small fw-bold text-muted">
                Bairro / Localidade *
              </label>
              <input type="text" id="bairro" class="form-control rounded-3" required
                     placeholder="Ex: Ferrugem, Centro...">
              <div class="invalid-feedback">Informe o bairro em Garopaba.</div>
            </div>
          </div>

          <div class="row g-2 mb-3">
            <div class="col-md-6">
              <label for="precoBase" class="form-label small fw-bold text-muted">
                Preço Base (R$) *
              </label>
              <input type="number" id="precoBase" class="form-control rounded-3" required
                     step="0.01" min="0" placeholder="Ex: 50.00">
              <div class="invalid-feedback">Informe um valor válido em R$.</div>
            </div>
            <div class="col-md-6">
              <label for="telefone" class="form-label small fw-bold text-muted">
                WhatsApp / Telefone *
              </label>
              <input type="tel" id="telefone" class="form-control rounded-3" required
                     pattern="[0-9]{10,11}" placeholder="48999998888">
```

```
            <div class="invalid-feedback">Informe com DDD (10 ou 11 dígitos).</div>
          </div>
        </div>

        <div class="mb-3">
          <label for="descricao" class="form-label small fw-bold text-muted">
            Descrição dos Serviços e Produtos *
          </label>
          <textarea id="descricao" class="form-control rounded-3" rows="3" required
                    placeholder="Descreva os produtos e diferenciais..."></textarea>
          <div class="invalid-feedback">Insira uma breve descrição sobre o negócio.</div>
        </div>
      </div>

      <div class="modal-footer border-0 pt-0">
        <button type="button" data-bs-dismiss="modal"
                class="btn btn-light rounded-pill px-4 text-muted fw-bold">Cancelar</button>
        <button type="submit" id="btnSalvarModal"
                class="btn btn-success rounded-pill px-4 font-outfit fw-bold shadow-sm">
          <i class="bi bi-check2-circle me-1"></i>
          <span id="btnSalvarTexto">Salvar na Vitrine</span>
        </button>
      </div>
    </form>
  </div>
 </div>
</div>
```

Checkpoint de Validação 1.2: Como Testar a Abertura do Modal

Atualize o navegador e clique no botão amarelo **"Divulgar Serviço"**:

1. O modal deve abrir de forma suave, centralizada e com bordas arredondadas (`.rounded-4`);
2. Verifique se todos os campos estão bem espaçados no corpo do modal;
3. Clique em **"Cancelar"** ou no botão **"X"** para confirmar o fechamento nativo.

Fase 1.3/5: Estilização Complementar (styles.css)

Responsabilidade Arquitetural: Definir variáveis de cores institucionais do IFSC Garopaba, aplicar tipografia Google Fonts (*Outfit* e *Plus Jakarta Sans*), o gradiente no cabeçalho e a elevação suave nos cards (`transform: translateY(-5px)`).

styles.css CSS

```css
/* styles.css */
:root {
  --ifsc-green: #059669;
  --ifsc-dark-green: #047857;
  --garopaba-blue: #0284c7;
  --bg-page: #f8fafc;
  --text-dark: #0f172a;
}
body {
  font-family: 'Plus Jakarta Sans', sans-serif;
  background-color: var(--bg-page);
  color: var(--text-dark);
}
.font-outfit { font-family: 'Outfit', sans-serif; }
.hero-banner {
  background: linear-gradient(135deg, #064e3b 0%, #065f46 50%, #047857 100%);
  color: #ffffff;
  padding: 2.5rem 0 2rem;
  margin-bottom: 2rem;
  border-bottom: 3px solid #10b981;
}
.vitrine-card {
  transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
  background: #ffffff;
}
.vitrine-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 15px 30px -10px rgba(0,0,0,0.12) !important;
}
.badge-counter {
  background: rgba(255,255,255,0.2);
  backdrop-filter: blur(5px);
  border: 1px solid rgba(255,255,255,0.3);
  padding: 0.4rem 0.9rem;
  border-radius: 50px;
  font-size: 0.85rem;
  font-weight: 700;
}
```

Checkpoint de Validação 1.3: Como Testar a Estilização

Salve `styles.css` na mesma pasta do `index.html` e atualize a página:

1. O banner superior exibe o **degradê verde** com tipografia branca;
2. O contador surge como uma pílula translúcida ao lado de **"Divulgar Serviço"**;
3. No Console (F12), o erro 404 de `styles.css` desaparece (o de `js/main.js` segue até o Passo 5).

Fase 2/5: Utilitários Puros (js/utils/formatters.js)

Responsabilidade Arquitetural: Funções puras e reutilizáveis de transformação de dados. Recebem entradas brutas e retornam formatos amigáveis para o usuário brasileiro (moeda e telefone), além de realizar a sanitização de strings contra ataques XSS (*Cross-Site Scripting*). Não tocam no DOM nem no armazenamento.

Croqui Visual 2: Fluxo Puro de Entrada e Saida

```
[ "48999998888" ] --------> [ formatarTelefone() ] --------> "(48) 99999-8888"
[ 35.5          ] --------> [ formatarMoeda()    ] --------> "R$ 35,50"
[ "<script>..." ] --------> [ escapeHtml()       ] --------> "&lt;script&gt;..."
```

js/utils/formatters.js — JavaScript

```javascript
// js/utils/formatters.js
export function formatarMoeda(valor) {
  const num = parseFloat(valor) || 0;
  return num.toLocaleString('pt-BR', {
    style: 'currency',
    currency: 'BRL'
  });
}

export function formatarTelefone(fone) {
  const digits = (fone || '').replace(/\D/g, '');
  if (digits.length === 11) {
    return digits.replace(
      /^(\d{2})(\d{5})(\d{4})/,
      '($1) $2-$3'
    );
  }
  return fone;
}

export function escapeHtml(texto) {
  const div = document.createElement('div');
  div.textContent = texto;
  return div.innerHTML;
}
```

Checkpoint de Validação 2: Teste via Console do Navegador (F12)

No Console do navegador (**F12**), execute:

JavaScript

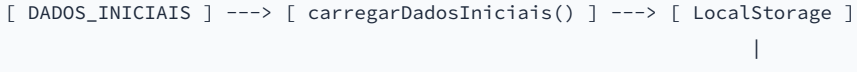
```
import('./js/utils/formatters.js').then(m => console.log(m.formatarMoeda(149.9)));
```

Resultado esperado: `"R$ 149,90"`. Comprova a exportação nomeada e a formatação nativa!

Fase 3/5: Camada de Serviços e Dados (js/services/vitrineService.js)

Responsabilidade Arquitetural: Isolar completamente a regra de persistência e ciclo de vida do CRUD (*Create, Read, Update, Delete*). Opera estritamente sobre a chave `'garopaba_vitrine_servicos'` do `localStorage` do navegador, sem gerar nenhuma tag HTML nem manipular o DOM.

Croqui Visual 3: Arquitetura da Camada de Servicos e Carga Inicial



js/services/vitrineService.js — JavaScript

```javascript
// js/services/vitrineService.js
const STORAGE_KEY = 'garopaba_vitrine_servicos';

// Dados modelo semente para enriquecer a experiência inicial
export const DADOS_INICIAIS = [
  {
    id: '1',
    nome: 'Maré Alta Artesanatos & Cerâmicas',
    categoria: 'Artesanato',
    bairro: 'Centro Histórico',
    precoBase: 35.00,
    telefone: '48991234567',
    descricao: 'Peças artesanais e utilitárias modeladas à mão ' +
      'com argila local e conchas de Garopaba.'
  },
  {
    id: '2',
    nome: 'Garopaba Web & Design Studio',
    categoria: 'Tecnologia',
    bairro: 'Ferrugem',
    precoBase: 150.00,
    telefone: '48998765432',
    descricao: 'Criação de websites profissionais responsivos, ' +
      'cardápios digitais e suporte para comércio local.'
  },
  {
    id: '3',
    nome: 'Pescado Fresco do Zequinha',
    categoria: 'Alimentação',
    bairro: 'Canto das Canoas',
    precoBase: 42.00,
    telefone: '48984561234',
    descricao: 'Peixes frescos e frutos do mar da pesca artesanal ' +
      'diária entregues com higiene e pontualidade.'
  }
```

```
];

// Função dedicada para carregar os dados modelo no LocalStorage
export function carregarDadosIniciais() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(DADOS_INICIAIS));
  return DADOS_INICIAIS;
}

// READ: Recupera os serviços ou carrega os dados modelo se vazio
export function obterServicos() {
  const dados = localStorage.getItem(STORAGE_KEY);
  if (!dados) {
    return carregarDadosIniciais();
  }
  try {
    return JSON.parse(dados);
  } catch (e) {
    console.error('Erro ao processar dados da vitrine:', e);
    return [];
  }
}

export function salvarServico(novo) {
  const servicos = obterServicos();
  const completo = {
    id: Date.now().toString(),
    dataCadastro: new Date().toISOString(),
    ...novo
  };
  servicos.unshift(completo);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return completo;
}

export function atualizarServico(id, dados) {
  const servicos = obterServicos();
  const idx = servicos.findIndex(s => s.id === id);
  if (idx !== -1) {
    servicos[idx] = { ...servicos[idx], ...dados,
                      dataEdicao: new Date().toISOString() };
    localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
    return servicos[idx];
  }
  return null;
}

export function removerServico(id) {
  const servicos = obterServicos().filter(s => s.id !== id);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return servicos;
}
```

Checkpoint de Validação 3: Teste de Persistência no Console (F12)

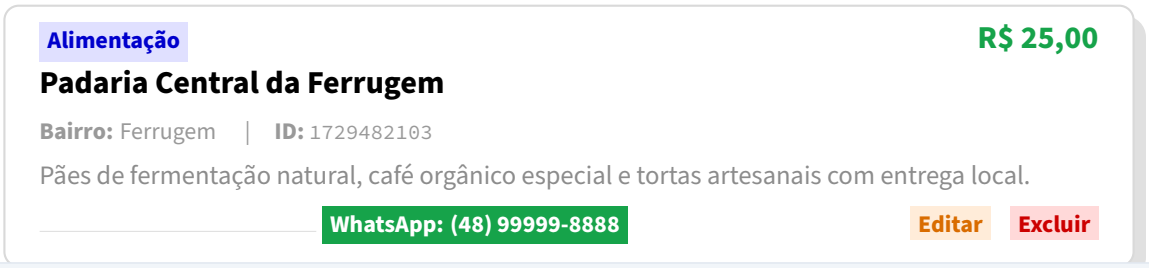
No Console (**F12**), teste a carga e a persistência dos dados:

```javascript
import('./js/services/vitrineService.js').then(s => {
  s.carregarDadosIniciais();
  console.log('Dados salvos:', s.obterServicos());
});
```

Resultado esperado: Array com os 3 serviços salvo em `garopaba_vitrine_servicos` (verifique também na aba **Application** → **Local Storage**).

Fase 4/5: Camada de Visão (js/views/vitrineView.js)

Responsabilidade Arquitetural: Construir e manipular os elementos visuais na interface. Transforma objetos de dados em cards HTML (`criarCardHtml`), renderiza coleções no contêiner (`renderizarCards`) e dispara notificações flutuantes (`exibirToast`).



js/views/vitrineView.js — JavaScript

```
// js/views/vitrineView.js
import { formatarMoeda, formatarTelefone, escapeHtml }
  from '../utils/formatters.js';

// Mapeamento de cores para categorias
const CORES_CATEGORIA = {
  'Alimentação': 'success',
  'Tecnologia': 'info',
  'Artesanato': 'warning',
  'Serviços Gerais': 'primary',
  'Turismo': 'secondary'
};

// Função para criar o HTML de um card de serviço
export function criarCardHtml(s) {
  const corBadge = CORES_CATEGORIA[s.categoria] || 'primary';
  const telNumeros = (s.telefone || '').replace(/\D/g, '');
  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card rounded-4 overflow-hidden">
        <div class="card-body p-4 d-flex flex-column">
          <div class="d-flex justify-content-between align-items-center mb-2">
            <span class="badge bg-${corBadge} px-3 py-2 rounded-pill font-outfit">
              <i class="bi bi-tag-fill me-1"></i>${escapeHtml(s.categoria)}
            </span>
            <span class="fw-bold text-success font-outfit fs-5">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>
          <h5 class="card-title fw-bold text-dark mt-2 mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-3">
            <i class="bi bi-geo-alt-fill text-danger me-1"></i>${escapeHtml(s.bairro)}
          </h6>
          <p class="card-text text-secondary small flex-grow-1" style="line-height: 1.5;">
            ${escapeHtml(s.descricao)}
          </p>
```

```
          <hr class="my-3 text-muted opacity-25">
          <div class="d-flex justify-content-between align-items-center mt-auto">
            <a href="https://wa.me/55${telNumeros}" target="_blank"
               class="btn btn-sm btn-outline-success rounded-pill px-3 fw-bold">
              <i class="bi bi-whatsapp me-1"></i>${formatarTelefone(s.telefone)}
            </a>
            <div class="d-flex gap-1">
              <button class="btn btn-sm btn-outline-primary btn-editar rounded-pill px-2"
                      data-id="${s.id}" title="Editar serviço">
                <i class="bi bi-pencil-square"></i>
              </button>
              <button class="btn btn-sm btn-outline-danger btn-excluir rounded-pill px-2"
                      data-id="${s.id}" title="Remover da vitrine">
                <i class="bi bi-trash"></i>
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>`;
}

// Renderização dinâmica dos cards e tratamento de estado vazio
export function renderizarCards(servicos, containerElement) {
  if (!servicos || servicos.length === 0) {
    containerElement.innerHTML = `
      <div class="col-12 text-center py-5">
        <div class="p-4 rounded-4 bg-light border">
          <i class="bi bi-inbox fs-1 text-muted d-block mb-2"></i>
          <h5 class="fw-bold text-secondary mb-1">Nenhum serviço cadastrado nesta categoria</h5>
          <p class="text-muted small mb-0">Cadastre um novo serviço ou altere o filtro acima.</p>
        </div>
      </div>`;
    return;
  }
  containerElement.innerHTML = servicos.map(criarCardHtml).join('');
}

// Notificações flutuantes assíncronas (Toasts) com auto-fechamento
export function exibirToast(mensagem, tipo = 'success') {
  const toastEl = document.getElementById('toastNotificacao');
  const toastMsg = document.getElementById('toastMensagem');
  const toastTitulo = document.getElementById('toastTitulo');

  if (toastEl && toastMsg) {
    toastMsg.innerText = mensagem;
    if (toastTitulo) {
      toastTitulo.innerText = tipo === 'success' ? 'Sucesso!' : 'Aviso';
    }
    toastEl.className = `toast align-items-center text-bg-${tipo} border-0 shadow-lg`;
    const toast = bootstrap.Toast.getOrCreateInstance(toastEl, { delay: 4000 });
```

```
    toast.show();
  }
}
```

Checkpoint de Validação 4: Teste do Toast no Console (F12)

No Console (**F12**), teste a notificação flutuante:

JavaScript

```
import('./js/views/vitrineView.js').then(v => v.exibirToast('Notificação funcionando!'));
```

Resultado esperado: O Toast verde surge no canto superior direito da tela.

Fase 5/5: Orquestrador Central (js/main.js)

Responsabilidade Arquitetural: O controlador principal da aplicação. Importa os serviços e as visões, escuta o carregamento do DOM, conecta os eventos de submissão do formulário (*Create & Update*), a delegação de cliques nos cards (*Edit & Delete*) e a reatividade instantânea dos filtros.

Croqui Visual 5: Ciclo de Vida da Orquestracao

```
Usuario clica/submete ---> main.js valida ---> vitrineService (localStorage)
                                                        |
Interface renderiza cards <--- vitrineView <----------+ (atualizarInterface)
```

js/main.js — JavaScript

```javascript
// js/main.js
import {
  obterServicos, salvarServico,
  atualizarServico, removerServico
} from './services/vitrineService.js';
import {
  renderizarCards, exibirToast
} from './views/vitrineView.js';

document.addEventListener('DOMContentLoaded', () => {
  const form = document.getElementById('formCadastro');
  const container = document.getElementById('vitrineContainer');
  const filtroCategoria = document.getElementById('filtroCategoria');
  const contadorEl = document.getElementById('totalServicosBadge');
  const modalEl = document.getElementById('modalCadastro');
  const servicoIdInput = document.getElementById('servicoId');

  function atualizarInterface() {
    const todos = obterServicos();
    const filtro = filtroCategoria.value;
    const filtrados = filtro === 'todas'
      ? todos
      : todos.filter(s => s.categoria === filtro);
    renderizarCards(filtrados, container);
    if (contadorEl) {
      contadorEl.innerText = `${todos.length} Serviços Cadastrados`;
    }
  }

  // Filtros Reativos
  filtroCategoria.addEventListener('change', atualizarInterface);
  atualizarInterface();

  // Reset do Modal ao abrir para novo cadastro
  if (modalEl) {
    modalEl.addEventListener('show.bs.modal', () => {
```

```
    if (!servicoIdInput.value) {
      form.reset();
      form.classList.remove('was-validated');
      const modalTitulo = document.getElementById('modalCadastroLabel');
      if (modalTitulo) {
        modalTitulo.innerHTML =
          '<i class="bi bi-shop text-success me-2"></i>Cadastrar Serviço na Vitrine';
      }
      const btnSalvarTexto = document.getElementById('btnSalvarTexto');
      if (btnSalvarTexto) {
        btnSalvarTexto.innerText = 'Salvar na Vitrine';
      }
    }
  });

  modalEl.addEventListener('hidden.bs.modal', () => {
    form.reset();
    servicoIdInput.value = '';
    form.classList.remove('was-validated');
  });
}

// Submissão do Formulário (Create & Update)
form.addEventListener('submit', (e) => {
  e.preventDefault();
  if (!form.checkValidity()) {
    e.stopPropagation();
    form.classList.add('was-validated');
    return;
  }

  const idAtual = servicoIdInput ? servicoIdInput.value : '';
  const getVal = (id) => document.getElementById(id).value.trim();

  const dados = {
    nome: getVal('nome'),
    categoria: document.getElementById('categoria').value,
    bairro: getVal('bairro'),
    precoBase: parseFloat(getVal('precoBase')) || 0,
    telefone: getVal('telefone'),
    descricao: getVal('descricao')
  };

  if (idAtual) {
    atualizarServico(idAtual, dados);
    exibirToast('Serviço atualizado com sucesso!');
  } else {
    salvarServico(dados);
    exibirToast('Empreendimento cadastrado com sucesso!');
  }
```

```
    form.reset();
    if (servicoIdInput) servicoIdInput.value = '';
    form.classList.remove('was-validated');

    if (modalEl) {
      const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
      modal.hide();
    }
    atualizarInterface();
  });

  // Delegação de Cliques nos Cards (Edit & Delete)
  container.addEventListener('click', (e) => {
    const btnEditar = e.target.closest('.btn-editar');
    if (btnEditar) {
      const id = btnEditar.getAttribute('data-id');
      const item = obterServicos().find(s => s.id === id);
      if (item) {
        if (servicoIdInput) servicoIdInput.value = item.id;
        document.getElementById('nome').value = item.nome;
        document.getElementById('categoria').value = item.categoria;
        document.getElementById('bairro').value = item.bairro;
        document.getElementById('precoBase').value = item.precoBase;
        document.getElementById('telefone').value = item.telefone;
        document.getElementById('descricao').value = item.descricao;

        const modalTitulo = document.getElementById('modalCadastroLabel');
        if (modalTitulo) {
          modalTitulo.innerHTML =
            '<i class="bi bi-pencil-square text-primary me-2"></i>Editar Serviço na Vitrine';
        }
        const btnSalvarTexto = document.getElementById('btnSalvarTexto');
        if (btnSalvarTexto) {
          btnSalvarTexto.innerText = 'Salvar Alterações';
        }

        if (modalEl) {
          const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
          modal.show();
        }
      }
      return;
    }

    const btnExcluir = e.target.closest('.btn-excluir');
    if (btnExcluir) {
      const id = btnExcluir.getAttribute('data-id');
      if (confirm('Deseja realmente remover este serviço da vitrine?')) {
        removerServico(id);
        atualizarInterface();
        exibirToast('Serviço removido da vitrine.', 'warning');
```

```
      }
    }
  });
});
```

Checkpoint de Validação 5: Teste Completo da Aplicação Ponta a Ponta

Aplicação 100% integrada! Execute o fluxo completo no navegador:

1. **Validação:** Clique em ”Divulgar” e salve vazio. Os campos ficam vermelhos e nada é salvo;
2. **Create:** Preencha e salve. O card surge na vitrine e o contador soma +1 (sob o Toast);
3. **Filter:** Mude a categoria no select. Apenas serviços da categoria escolhida permanecem;
4. **Update:** Clique no lápis em um card, altere os dados no modal e salve;
5. **Delete:** Clique na lixeira, confirme o aviso e veja o item ser removido imediatamente.

7 Checklist de Conclusão e Critérios Avaliativos

Antes de realizar o commit final no repositório, certifique-se de validar todos os itens:

Antes de entregar, confira se...

- [ ] O projeto utiliza estritamente o padrão ES6 Modules com a tag: `<script type="module" src="js/main.js"></script>`;
- [ ] A estrutura em pastas (`js/services/`, `js/views/`, `js/utils/`) foi rigorosamente respeitada;
- [ ] O arquivo `vitrineService.js` opera puramente sobre o `localStorage` sem manipular elementos do DOM;
- [ ] As quatro operações do CRUD (**Create, Read, Update e Delete**) foram implementadas e testadas com persistência no navegador;
- [ ] O formulário exibe validações visuais nativas do Bootstrap 5 (`.was-validated` e `.invalid-feedback`) quando campos obrigatórios não são preenchidos;
- [ ] A vitrine reage instantaneamente aos filtros de categoria sem recarregar a página;
- [ ] O código está devidamente publicado e em pleno funcionamento no **GitHub Pages**.